

Spring

What is Maven?

- Maven is Build Management Tool.
- Build means .java file gets compiled into .class file and then packaged into .jar / .war.

Building a software project typically consists of such tasks as

- Downloading dependencies
- Putting additional jars on a classpath, compiling source code into binary code
- Running tests
- Packaging compiled code into deployable artifacts such as JAR, WAR, and ZIP files
- Deploying these artifacts to an application server or repository.

- Maven will do all above points automatically for us.

What is Spring Framework?

- It is a Framework of Frameworks.
- It helps developers to work their application rather than worrying about non-functional code.
- In simple way it will help to focus on our Business Logic rather than Worrying about other non-functional requirements.
- It minimizes boilerplate Java code.
- **Spring Framework is a Dependency Injection Framework to make Java application Loosely Coupled.**
- Spring provides IOC (Inversion Of Control) which helps Dependency Injection.

What is Dependency Injection?

- It is a Design Pattern.
- It injects one object into another object.
- Simply means one object dependency will be injected into another Object.
- It is used to achieve loose coupling in Java.
- We can achieve Dependency Injection by 3 Ways → Setter Method and Constructor Method and Field Level.

Tightly Coupled Classes:	Loosely Coupled Classes:
→ Tightly coupled classes have a strong dependency, where one class directly depends on another. → For example: <pre>class Engine { public void start() { // Start the engine } } public class Car { private Engine engine; public Car() { engine = new Engine(); // Tightly coupled to Engine class } public void start() { engine.start(); } }</pre>	→ Loosely coupled classes minimize dependencies and make the code more maintainable and flexible. → For example: <pre>public interface Engine { void start(); } public class ElectricEngine implements Engine { public void start() { // Start an electric engine } } public class PetrolEngine implements Engine { public void start() { // Start a petrol engine } } public class Car { private Engine engine; public Car(Engine engine) { this.engine = engine; // Loosely coupled to Engine class } public void start() { engine.start(); } }</pre>

Inversion Of Control (IOC)

- Create the Object.
- Hold them in memory.
- And those Object Inject them in Another Object as Require.
- Complete Life cycle of Object creation to destruction is maintained by IOC Container.
- We need provide Beans information and Configuration Information to IOC.

What is Spring Container?

- Core Component(Heart) Like JVM
- Responsibilities
 - Manage bean object
 - Manage bean life cycle
 - Dependency injection
 - AOP
 - Transaction Manager

Type – BeanFactory (Old) – Lazy Loading – when call getBean()

ApplicationContext (New) – Interface – Eager Loading – when application loading

ApplicationContext

- It represent bean container
- ApplicationContext (New) – Interface – Eager Loading – when application loading
- It extend BeanFactory
 - ClassPathXMLApplicationContext
 - AnnotationConfigApplicationContext
 - FileSystemXMLApplicationContext

SetterInjection - <property> Tag

- Create Some Java Pojo **with Setter Method**
- Create XML file for Configuration which will provide instruction to spring container with name as config.xml
- Add in this xml file <bean> tag
- For Example –

Primitive Type Injection

If we have **Student.class**

Then add this tag in XML **config.xml** file.

```
<bean class="com.example.Student" name="student">
    <property name="studentName" value = "ABC" />
</bean>
```

In main class write functionality for using this bean

```
ApplicationContext context = new ClassPathXMLApplicationContext("config.xml");
Student student = (Student) context.getBean("student");
sout(student);
```

Reference Type Injection

If we have **Student.class** and **Address.class**

Then add this tag in XML **config.xml** file.

```
<bean class="com.example.Student" name="student">
    <property name="studentName" value = "ABC" />
    <property name="studentAddress" ref = "address" />
</bean>
<bean class="com.example.Address" name="address">
    <property name="city" value = "XYZ city">
```

`</bean>`

In main class write functionality for using this bean

```
ApplicationContext context = new ClassPathXMLApplicationContext("config.xml");
Student student = (Student) context.getBean("student");
sout(student);
sout(student.getAddress());
```

Constructor Injection - `<constructor-arg>` Tag

- Create Some Java Pojo **with Contructor**.
- Create XML file for Configuration which will provide instruction to spring container with name as config.xml
- Add in this xml file `<bean>` tag
- For Example –

Primitive Type Injection

If we have **Student.class**

Then add this tag in XML **config.xml** file.

```
<bean class="com.example.Student" name="student">
    < constructor-arg name="studentName" value = "ABC" />
</bean>
```

In main class write functionality for using this bean

```
ApplicationContext context = new ClassPathXMLApplicationContext("config.xml");
Student student = (Student) context.getBean("student");
sout(student);
```

Reference Type Injection

If we have **Student.class** and **Address.class**

Then add this tag in XML **config.xml** file.

```
<bean class="com.example.Student" name="student">
    < constructor-arg name="studentName" value = "ABC" />
    < constructor-arg name="studentAddress" ref = "address" />
</bean>
<bean class="com.example.Address" name="address">
    < constructor-arg name="city" value = "XYZ city">
</bean>
```

In main class write functionality for using this bean

```
ApplicationContext context = new ClassPathXMLApplicationContext("config.xml");
Student student = (Student) context.getBean("student");
sout(student);
sout(student.getAddress());
```

Ambiguity Problem in with Constructor Injection

If we have class with method overloading constructor

For Example

```
Public void add(int a, int b)
{
    System.out.println(a+b);
}
Public void add(double a, double b)
{
    System.out.println(a+b);
}
```

Now we configure the bean in config.xml file

```
<bean class="com.example.add" name="add">
    < constructor-arg value = "10">
    < constructor-arg value = "20">
</bean>
```

Now Here we get Ambiguity because it will search first match which int method but we have two method it can get confused.

Now we need to prevent this with "type" argument.

```
<bean class="com.example.add" name="add">
    < constructor-arg value = "10" type="int">
    < constructor-arg value = "20" type="int">
</bean>
```

Life Cycle of Spring Bean

- Loading Bean Definition
- Bean Object Instantiation
- Bean Initialization
- Bean Destruction

Scope of Bean: - **@Scope** – It will tell scope of the bean means if we want to declare the scope of the Bean.

In that commonly we use singleton or prototype.

Type of Scope

Singleton – by default scope of spring bean – it will return same object reference every time in entire application.

Prototype – it will return always new object reference – it will create new object every time

Request – for HTTP request objects – it will create new instance for each HTTP request.

Session – for only session objects – it will create single instance of the bean per web socket session

Globalsession

Spring Expression Language – SpEL

It supports Parsing and executing expression with the help of @Value annotation.

For Example - @Value("#{Expression}")

@Value("#{11 + 22}") – it will return 33

@Value("#{1 > 0}") – it will return True.

Spring JDBC

Spring JDBC is a powerful mechanism to connect to the database and execute SQL queries.

JDBC is API to perform operations with Database.

Spring MVC – Model View Controller

It is sub framework of Spring Framework which is used to build a [Web Application](#).

It is built on Top of [Servlet API](#).

It follows [Model – View – Controller](#) Design pattern.

Spring MVC Interceptor

In simple word interceptor works process some operation in between sending the Request.

Stereotype Annotation

@Component Annotation

First we need to create POJO class with setter and getter and constructor.

Class Student

```
{  
    \\ Statements getter and setters  
    \\ toString Method  
}
```

Now we create configuration file

Create XML file for Configuration which will provide instruction to spring container with name as config.xml

Config.xml

```
<context:component-scan base-package = "com.springcore.stereotype" />
```

After that we need to write annotation on class

@Component

Class Student

```
{  
    @Value("Set Values for fields")  
    \\ Statements getter and setters  
    \\ toString Method  
}
```

Advantages of Spring Framework

1. Modular and Lightweight:

=> Spring follows a modular architecture, allowing developers to use only the components they need, making applications more lightweight and easier to maintain. This modularity also promotes better code organization.

2. Flexible Configuration:

=> Spring supports multiple configuration options, including XML, Java-based configuration, and annotation-based configuration. This flexibility allows developers to choose the most suitable approach for their projects.

3. Dependency Injection (DI):

=> Spring supports DI, which simplifies the management of component dependencies, making code more testable and adaptable to changes.

4. Aspect-Oriented Programming (AOP):

=> Spring provides AOP support, allowing developers to separate cross-cutting concerns like logging, security, and transactions from the core application logic. This improves code modularity and maintainability.

5. Simplified Database Access:

=> Spring's JDBC and Object-Relational Mapping (ORM) support (e.g., Hibernate, JPA) simplifies database access, reduces boilerplate code, and improves data access efficiency.

6. Testing Support:

=> Spring's architecture encourages writing unit tests, and it provides support for integration testing.

7. Security:

=> Spring Security provides a robust framework for implementing authentication and authorization, making it easier to secure web applications and services.

8. Integration Capabilities:

=> Spring provides integration with various technologies and frameworks, such as angular, react, messaging systems (JMS), web services (SOAP and REST), and other third-party libraries and APIs.

9. Scalability:

=> Spring applications can be designed for scalability and can easily integrate with cloud-native technologies and microservices architectures.

10. Open Source:

=> Spring is open-source, which means it's free to use and can be customized to meet specific project requirements.

